

FinRelief AI: AI-Powered Debt Relief & Financial Recovery Platform

Project Description:

FinRelief AI is an intelligent, AI-powered web application that simplifies and automates debt management and financial recovery for borrowers. It lets users manage loan details, analyze their financial health, and receive both rule-based settlement recommendations and AI-generated negotiation letters, all through a secure, authenticated web interface.

The backend is built with FastAPI, SQLAlchemy, and SQLite, with authentication handled via bcrypt password hashing and JWT bearer tokens. A deterministic financial engine computes EMI-to-income ratio, monthly surplus, a weighted debt-stress score, and a recommended settlement percentage for every loan. The frontend is a React + Vite single-page application that consumes this API to present a login/registration flow, a loan management screen, a financial dashboard, and an AI negotiation page. When available, the Google Gemini API (gemini-1.5-flash) drafts a lender-specific negotiation letter from the borrower's live financial data; if no API key is configured or the call fails, a deterministic rule-based template takes over automatically so the feature never breaks.

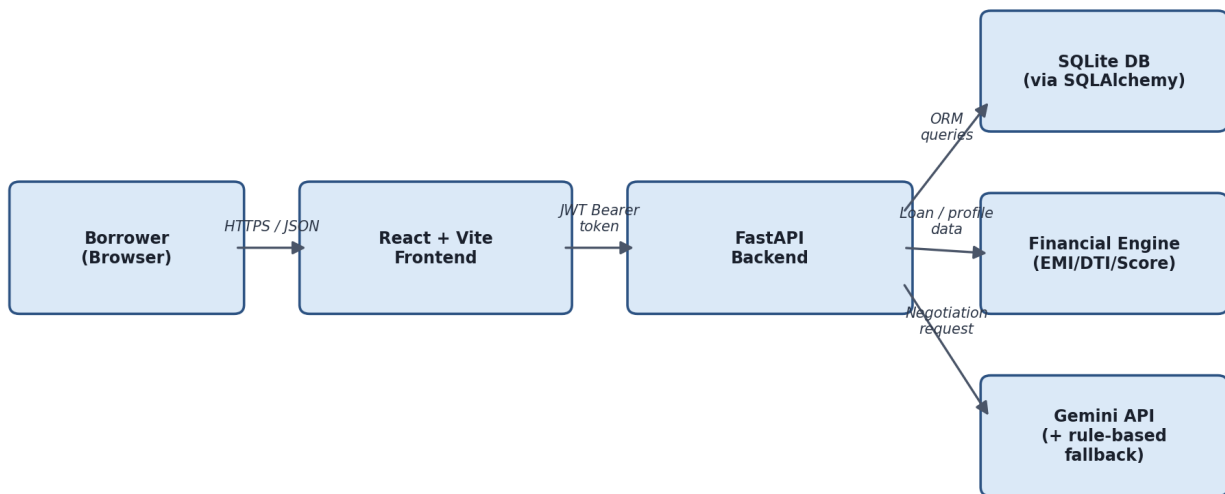
Scenarios

Scenario 1: A borrower registers, adds a loan (lender, outstanding balance, EMI, overdue months) along with their monthly income. On submission, the backend immediately computes the EMI-to-income ratio, monthly surplus, debt-stress score, risk category, and a recommended settlement percentage, and returns them to the dashboard.

Scenario 2: A borrower with a loan flagged as high-stress clicks "Negotiate." The backend builds a structured prompt from the loan's live financial data and calls the Gemini API to draft a lender-specific negotiation letter; if Gemini is unavailable, a rule-based fallback letter is generated instead so the borrower always receives a usable result.

Scenario 3: A borrower opens the financial dashboard to review monthly surplus, EMI ratio, debt-stress level, and settlement history across all their loans, and browses past AI-generated negotiation letters to track their recovery progress over time.

Technical Architecture:



Description: FinRelief AI uses a three-tier architecture. The React + Vite frontend collects borrower input (registration, loan details, negotiation requests) and communicates with the backend over HTTPS/JSON, attaching a JWT bearer token to every authenticated request. The FastAPI backend validates the token, applies per-user data scoping at the query level, and routes requests to either the financial engine (pure Python EMI/DTI/settlement scoring against SQLite via SQLAlchemy) or the Gemini negotiation service (with an automatic rule-based fallback). All persistent data - users, loans, settlement records, and negotiation history - is stored in SQLite through the SQLAlchemy ORM, with DATABASE_URL configurable via environment variable for future migration to a production RDBMS.

(Note: An ER Diagram covering Users, Loans, SettlementRecord, and NegotiationHistory has been submitted separately as part of the database design deliverable.)

Pre-requisites:

- **FastAPI & Python Knowledge:** FastAPI Documentation
- **SQLAlchemy ORM Familiarity:** SQLAlchemy Documentation
- **React & Vite Skills:** React Documentation, Vite Documentation
- **Google Gemini API Familiarity:** Gemini API Documentation
- **JWT & bcrypt Authentication Concepts:** JWT Introduction
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Python 3.8+, Node.js 18+, npm

Project Workflow:

Milestone 1: Application Development & System Setup

- **Activity 1.1:** Set up the FastAPI backend project structure (main.py, database.py, models.py, schemas.py, auth.py, routers package) and the React + Vite frontend project.
- **Activity 1.2:** Configure SQLAlchemy with SQLite and define the DATABASE_URL environment variable for future portability to a production database.
- **Activity 1.3:** Implement bcrypt-based password hashing and JWT (HS256, 24-hour expiry) issuing/validation via auth.py and get_current_user.
- **Activity 1.4:** Install and pin dependencies (fastapi, uvicorn, sqlalchemy, bcrypt, python-jose, google-generativeai) in requirements.txt.

Milestone 2: AI Integration & Financial Processing Setup

- **Activity 2.1:** Build financial_engine.py - EMI-to-income ratio, monthly surplus, a weighted debt-stress score (50% EMI burden / 30% overdue months / 20% surplus), and risk categorization (Low/Moderate/High/Critical).
- **Activity 2.2:** Implement the settlement-percentage formula, starting from a risk-based base (85/70/55/40%), reduced for longer overdue periods, and floored at 25% of principal.
- **Activity 2.3:** Build gemini_service.py to call the Gemini API for negotiation-letter drafting, with a deterministic _fallback_letter() used whenever the API key is unset or the call fails.

Milestone 3: Database Management & Financial Data Storage Setup

- **Activity 3.1:** Define SQLAlchemy models - User, Loan, SettlementRecord, NegotiationHistory - with relationships scoped to the owning user.
- **Activity 3.2:** Ensure every query in loans_router.py, financial_router.py, and negotiation_router.py filters by user_id == current_user.id, enforcing per-user data isolation at the database layer.

Milestone 4: Frontend Integration & User Interface Development

- **Activity 4.1:** Build the Login/Register pages and AuthContext for JWT-based session state.
- **Activity 4.2:** Build the Loans page (CRUD + trigger analysis), Financial Dashboard (EMI/DTI/surplus/score visualizations), and the AI Negotiation page (generate + history).
- **Activity 4.3:** Wire all pages to the FastAPI backend via a shared API client that attaches the JWT bearer token to every request.

Milestone 5: Testing, Debugging & Performance Optimization

- **Activity 5.1:** Run end-to-end smoke tests across the full flow: register → login → add loan → analyze → view dashboard → generate negotiation letter.
- **Activity 5.2:** Diagnose and fix a passlib/bcrypt version conflict that was silently breaking registration, by calling bcrypt directly instead of through passlib.
- **Activity 5.3:** Diagnose and fix an unrealistic DTI-ratio calculation (it was comparing total loan balance to monthly income instead of annual income), then re-validate the corrected financial-health score.

Milestone 6: Version Control, Project Finalization & Deployment Readiness

- **Activity 6.1:** Clean up dev artifacts (stray .db files, caches) and organize the backend into a modular structure (routers/, models.py, schemas.py, financial_engine.py, gemini_service.py).
- **Activity 6.2:** Write the top-level README with setup instructions for both servers, and a .env.example documenting required environment variables.
- **Activity 6.3:** Prepare the project for GitHub submission and (optional) cloud deployment.

Milestone 1: Application Development & System Setup

This milestone establishes the foundational architecture for FinRelief AI - the FastAPI backend project structure, the SQLAlchemy database connection, and JWT-based authentication.

Activity 1.1: Backend Project Structure

The backend is organized into clearly separated modules, each with a single responsibility:

```
backend/  
  main.py           - FastAPI app entrypoint, CORS, router registration  
  database.py       - SQLAlchemy engine/session setup (DATABASE_URL from env)  
  models.py         - User, Loan, SettlementRecord, NegotiationHistory  
  schemas.py        - Pydantic request/response schemas  
  auth.py           - bcrypt hashing, JWT issue/validate, get_current_user  
  financial_engine.py - EMI/DTI/settlement scoring logic  
  gemini_service.py - Gemini API integration + rule-based fallback  
  routers/  
    auth_router.py   - /api/auth/register, /api/auth/login  
    loans_router.py  - full CRUD, ownership-scoped  
    financial_router.py - /analyze, /dashboard  
    negotiation_router.py - /generate, /history
```

Activity 1.3: Authentication

Passwords are hashed with bcrypt (with a defensive 72-byte truncation guard) before storage. On successful login, the backend issues a signed JWT access token (HS256, 24-hour expiry). The frontend stores this token and attaches it as a Bearer token on every subsequent API call; FastAPI's OAuth2PasswordBearer combined with a get_current_user dependency validates the token and resolves the requesting user on each protected route.

Milestone 2: AI Integration & Financial Processing Setup

Activity 2.1: Financial Scoring Logic

financial_engine.py computes, for every loan, the EMI-to-income ratio, monthly surplus, and a weighted debt-stress score. The score blends three signals - EMI burden (50% weight), overdue duration (30% weight), and remaining cash-flow surplus (20% weight) - and maps the result onto a Low / Moderate / High / Critical risk category. A recommended settlement percentage is then derived from a risk-based base rate, reduced for longer overdue periods and floored at 25% of the outstanding principal so recommendations never suggest an unrealistically low settlement.

Activity 2.3: AI Negotiation with Automatic Fallback

gemini_service.py builds a structured prompt from the borrower's live loan and financial-profile data and calls the Gemini API (gemini-1.5-flash) to draft a professional, lender-specific negotiation letter. If GEMINI_API_KEY is unset, or the API call raises an exception for any reason, the service transparently falls back to a deterministic template-based letter generator, so the negotiation feature always returns a usable result to the borrower.

Milestone 3: Database Management & Financial Data Storage Setup

Every loan, settlement, and negotiation-history query is scoped with user_id == current_user.id, so access control is enforced at the database-query level rather than through a separate role system - a borrower can only ever read, update, or delete their own records. DATABASE_URL is read from an environment variable, so the same SQLAlchemy models and routers can point at PostgreSQL or another production RDBMS later with no changes to business logic.

Milestone 4: Frontend Integration & User Interface Development

The React + Vite frontend is a single-page application with dedicated routes for each stage of the borrower journey: Login/Register, Loans (add/edit/delete + trigger analysis), Financial Dashboard (EMI ratio, DTI ratio, monthly surplus, and a visual health-score gauge), and AI Negotiation (generate a letter and browse negotiation history). AuthContext holds the JWT in React state and a shared API client attaches it as a Bearer token to every request, redirecting to the login page if a request comes back unauthorized.

Milestone 5: Testing, Debugging & Performance Optimization

The full user flow was verified end-to-end against the real HTTP API: register → login → add loan → view dashboard → generate settlement prediction → generate negotiation letter. Two real defects were found and fixed during this process:

- A passlib/bcrypt version incompatibility was silently breaking user registration; resolved by calling bcrypt directly instead of routing through passlib.
- The debt-to-income ratio formula was comparing total loan balance against monthly (rather than annual) income, producing nonsensical 500%+ figures; corrected to a standard annualized comparison and re-validated against sample loan data (see the Performance Testing document for full test results).

Milestone 6: Version Control, Project Finalization & Deployment Readiness

Development artifacts (stray SQLite database files created during testing, caches) were cleaned from the repository before packaging. The project ships with a top-level README covering setup for both servers, a .env.example documenting required environment variables (JWT_SECRET_KEY, GEMINI_API_KEY, DATABASE_URL), and a modular backend structure that keeps deployment configuration (CORS origins, environment-driven database URL) separate from business logic.

Exploring the Website's Web Pages:

Login / Register Page

Description: The entry point to FinRelief AI. New borrowers register with name, email, and password (hashed with bcrypt before storage); returning borrowers log in to receive a JWT access token, which the frontend stores and uses to authenticate every subsequent request.

Loans Page

Description: Lets a borrower add, edit, and delete loan records (lender, outstanding balance, EMI, overdue months). Adding or updating a loan immediately triggers the financial engine, which computes and persists a settlement-prediction record in the same request.

Financial Dashboard

Description: Aggregates every loan a borrower owns into portfolio-level totals - total outstanding balance, total EMI, EMI-to-income ratio, monthly surplus, debt-stress score, and risk category - alongside a visual financial-health gauge, so borrowers can see their overall position at a glance.

AI Negotiation Page

Description: Lets a borrower select a loan needing attention and generate a negotiation letter. The page shows the AI-drafted (or rule-based fallback) letter along with a running history of previously generated negotiation letters for that account.

Conclusion:

FinRelief AI is a full-stack, AI-assisted debt-recovery platform built with FastAPI, SQLAlchemy, SQLite, and React. It combines a transparent, rule-based financial engine with generative-AI negotiation support (backed by a reliable rule-based fallback) to help borrowers understand their debt position and take a concrete next step toward settlement. The project covered backend API design, JWT authentication, financial-domain scoring logic, third-party AI integration, and a responsive React frontend, with two real defects found and fixed through end-to-end testing along the way. Looking ahead, the platform could be extended with role-based admin oversight, migration to a production database, proactive stress-level alerts, and lender-specific negotiation letter templates.